

Esercizio: E-Commerce a Microservizi

1. Obiettivo

Realizzare un'applicazione web per la gestione di un piccolo **E-Commerce**, progettata secondo un'architettura a microservizi.

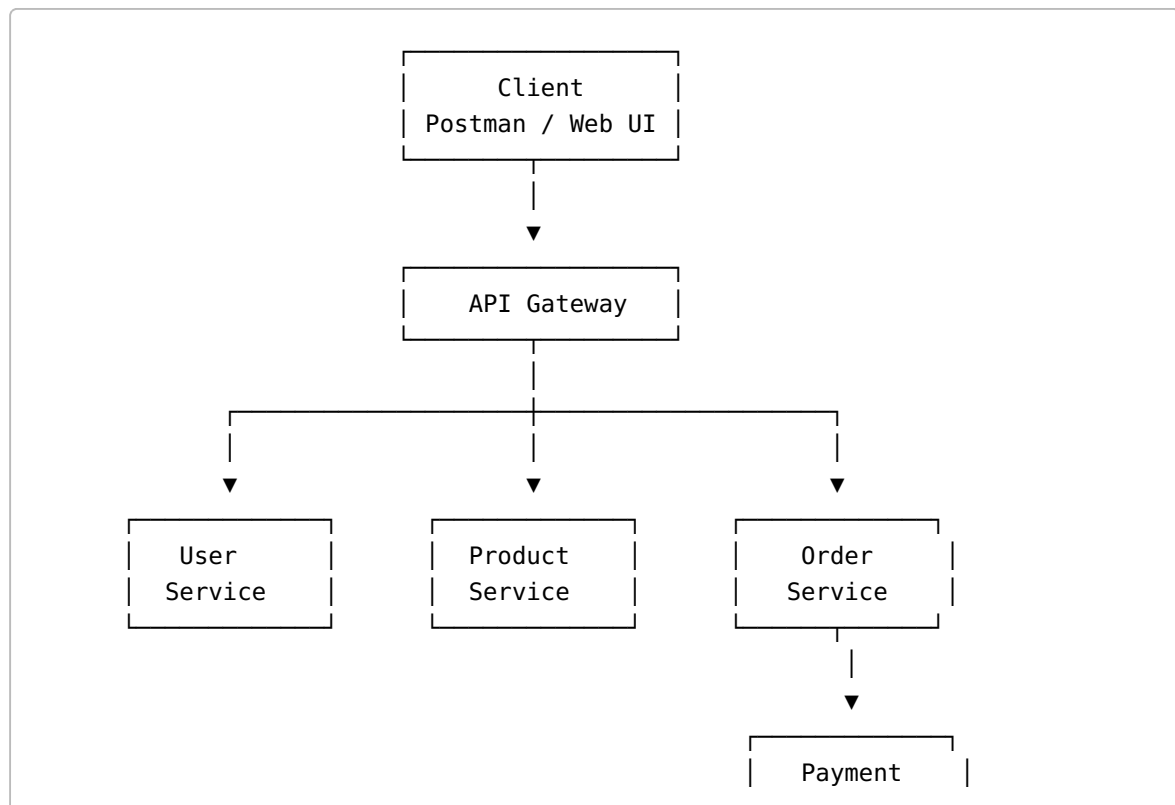
L'applicazione dovrà permettere di:

- registrare clienti;
- visualizzare prodotti;
- creare ordini;
- verificare la disponibilità dei prodotti;
- effettuare il pagamento di un ordine;
- inviare una notifica al cliente;
- registrare i log dell'applicazione;
- eseguire l'intero sistema tramite Docker;
- distribuire i microservizi su Kubernetes.

L'applicazione dovrà essere sviluppata utilizzando **Java + Spring Boot**.

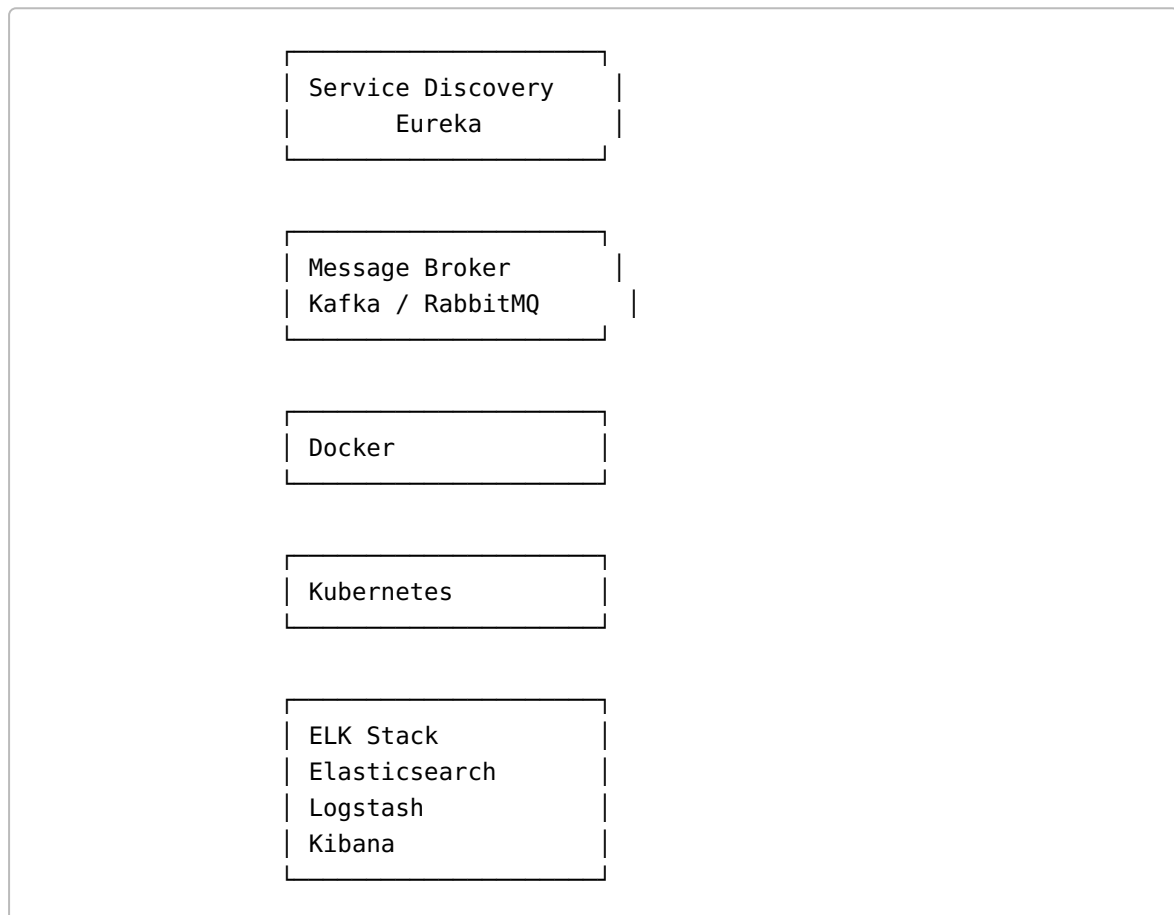
2. Architettura

Il sistema sarà composto inizialmente dai seguenti microservizi:





Successivamente verranno aggiunti:



3. Microservizi

User Service

Responsabile della gestione degli utenti.

Funzioni

POST	/users
GET	/users

```
GET    /users/{id}
PUT    /users/{id}
DELETE /users/{id}
```

Utente

```
{
  "id": 1,
  "name": "Antonio",
  "surname": "De Santis",
  "email": "antonio@email.it"
}
```

4. Product Service

Gestisce il catalogo dei prodotti.

API

```
POST   /products
GET    /products
GET    /products/{id}
PUT    /products/{id}
DELETE /products/{id}
```

Prodotto

```
{
  "id": 10,
  "name": "Laptop",
  "description": "Laptop 15 pollici",
  "price": 899.99,
  "stock": 20
}
```

Il Product Service deve inoltre permettere di:

```
GET /products/{id}/availability
```

per verificare se un prodotto è disponibile.

5. Order Service

È il microservizio centrale del progetto.

Permette di creare e gestire gli ordini.

API

```
POST /orders
GET  /orders
GET  /orders/{id}
```

Esempio:

```
{
  "userId": 1,
  "products": [
    {
      "productId": 10,
      "quantity": 2
    }
  ]
}
```

Quando viene creato un ordine, il servizio dovrà:

1. verificare che l'utente esista;
2. verificare la disponibilità dei prodotti;
3. calcolare il totale;
4. creare l'ordine;
5. richiedere il pagamento;
6. inviare una notifica.

6. Payment Service

Gestisce il pagamento.

Per questo esercizio **non bisogna utilizzare un vero sistema di pagamento.**

Creare invece un sistema simulato.

API:

```
POST /payments
GET /payments/{id}
```

Esempio:

```
{
  "orderId": 100,
  "amount": 1799.98
}
```

Il servizio restituisce:

```
{
  "paymentId": 500,
  "orderId": 100,
  "status": "SUCCESS"
}
```

Per rendere l'esercizio interessante, prevedere anche:

```
SUCCESS
FAILED
```

7. Notification Service

Questo microservizio simula l'invio di una notifica.

Non è necessario inviare realmente una mail.

Quando riceve l'evento:

```
ORDER_CREATED
```

deve produrre un log simile a:

```
INFO NotificationService:
Invio notifica per ordine 100
```

Successivamente potrà essere esteso per simulare:

EMAIL
SMS
PUSH

8. Comunicazione tra microservizi

Inizialmente utilizzare comunicazione **sincrona REST**.

Esempio:

```
graph TD
    OS[Order Service] -- HTTP --> PS[Product Service]
```

e:

```
graph TD
    OS[Order Service] -- HTTP --> US[User Service]
```

Successivamente modificare il progetto introducendo la comunicazione **asincrona**.

Ad esempio:

```
graph TD
    OS[Order Service] -- ORDER_CREATED --> MB[Message Broker]
    MB --> NS[Notification Service]
    MB --> PS[Payment Service]
```

In questo modo potrai confrontare:

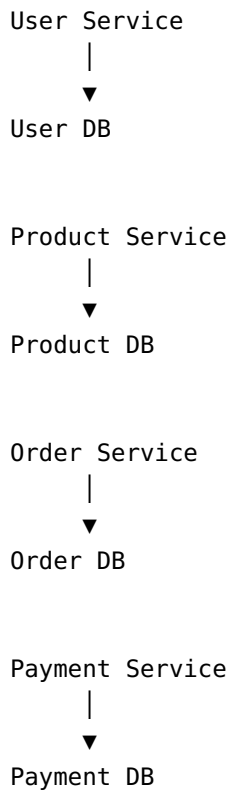
REST
vs
Comunicazione asincrona

9. Database

Ogni microservizio dovrà avere il proprio database.

NON creare un unico database condiviso.

Architettura:



Per esempio puoi utilizzare:

PostgreSQL

oppure iniziare con:

H2

durante lo sviluppo.

10. Spring Boot

Ogni microservizio deve essere un progetto Spring Boot indipendente.

Esempio:

```
ecommerce/  
├── user-service/  
├── product-service/  
├── order-service/  
├── payment-service/  
├── notification-service/  
├── api-gateway/  
└── service-discovery/
```

Ogni servizio deve avere:

```
Controller  
Service  
Repository  
Entity  
DTO  
Exception  
Configuration
```

quando necessario.

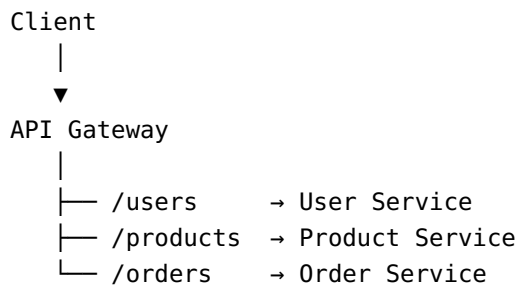
11. API Gateway

Creare un API Gateway che rappresenti il punto di ingresso dell'applicazione.

Il client non dovrà chiamare direttamente:


```
user-service
product-service
order-service
```

ma:



12. Service Discovery

Introdurre successivamente **Eureka**.

I microservizi non dovranno più conoscere necessariamente l'indirizzo fisico degli altri servizi.

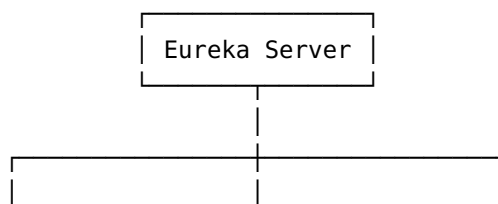
Ad esempio:

```
ORDER-SERVICE
PRODUCT-SERVICE
USER-SERVICE
PAYMENT-SERVICE
NOTIFICATION-SERVICE
```

si registrano presso:

```
EUREKA SERVER
```

Architettura:



▼
User Service

▼
Product Service

▼
Order Service

13. Gestione degli errori

Implementare una gestione centralizzata delle eccezioni.

Esempi:

```
UserNotFoundException  
ProductNotFoundException  
InsufficientStockException  
OrderNotFoundException  
PaymentFailedException
```

Restituire risposte HTTP appropriate.

Per esempio:

```
404 NOT FOUND
```

```
{  
  "status": 404,  
  "message": "Product not found",  
  "timestamp": "2026-09-07T10:30:00"  
}
```

14. Docker

Ogni microservizio dovrà avere il proprio:

```
Dockerfile
```

Esempio:

```
user-service  
  |  
  ▼  
  Docker Image  
  |
```

▼
Docker Container

Alla fine il sistema dovrà poter essere avviato tramite:

```
docker compose up
```

con:

```
API Gateway  
User Service  
Product Service  
Order Service  
Payment Service  
Notification Service  
Eureka  
Database  
Message Broker
```

15. Docker Compose

Creare un:

```
docker-compose.yml
```

che permetta di avviare l'intera applicazione.

Obiettivo:

```
docker compose up -d
```

e il sistema deve partire.

Per fermarlo:

```
docker compose down
```

16. Logging

Ogni microservizio deve produrre log significativi.

Esempio:

```
INFO OrderService - Creazione ordine
INFO OrderService - Verifica disponibilità prodotto
INFO ProductService - Prodotto 10 disponibile
INFO PaymentService - Pagamento ordine 100
INFO NotificationService - Notifica inviata
```

Successivamente configurare:

```
Elasticsearch
Logstash
Kibana
```

per centralizzare i log.

Architettura:

```
Microservices
|
| logs
▼
Logstash
|
▼
Elasticsearch
|
▼
Kibana
```

L'obiettivo finale è poter aprire Kibana e cercare, ad esempio:

```
orderId:100
```

e vedere tutti i log relativi all'ordine.

17. Kubernetes

Quando l'applicazione funziona con Docker Compose, effettuare la migrazione a Kubernetes.

Creare per ogni microservizio:

Deployment
Service

Esempio:

```
user-service-deployment.yaml
user-service-service.yaml

product-service-deployment.yaml
product-service-service.yaml

order-service-deployment.yaml
order-service-service.yaml
```

e così via.

18. Kubernetes Deployment

Un microservizio dovrà essere eseguito tramite un Deployment.

Esempio concettuale:

```
Kubernetes
|
▼
Order Deployment
|
├─ Pod Order #1
├─ Pod Order #2
└─ Pod Order #3
```

Impostare almeno:

```
replicas: 3
```

per `order-service`.

In questo modo potrai sperimentare la scalabilità.

19. Configurazione

Evitare di inserire nei sorgenti configurazioni come:

```
database host  
password  
porta  
URL di altri servizi
```

Utilizzare invece:

```
application.yml  
Environment Variables  
ConfigMap  
Secret
```

in Kubernetes.

20. Test

Creare test per i microservizi.

Almeno:

Unit Test

Testare:

```
Service  
Business Logic  
Controller
```

Integration Test

Verificare la comunicazione con:

```
Database  
REST API
```

Test dell'intero flusso

Testare il caso:

Creazione utente
↓
Creazione prodotto
↓
Creazione ordine
↓
Controllo disponibilità
↓
Pagamento
↓
Notifica

21. Scenario finale

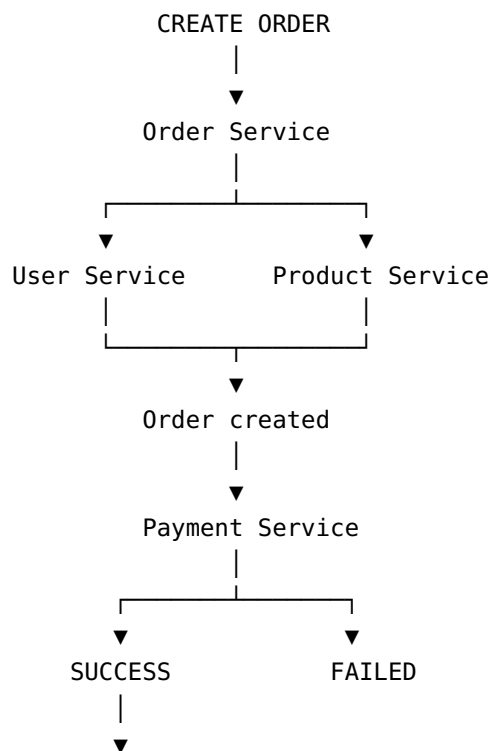
Il caso d'uso principale dovrà essere:

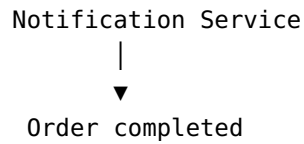
Acquisto di un prodotto

Il cliente esegue:

`POST /orders`

Il sistema deve eseguire:





22. Obiettivo finale

Al termine dell'esercizio dovrai essere in grado di eseguire:

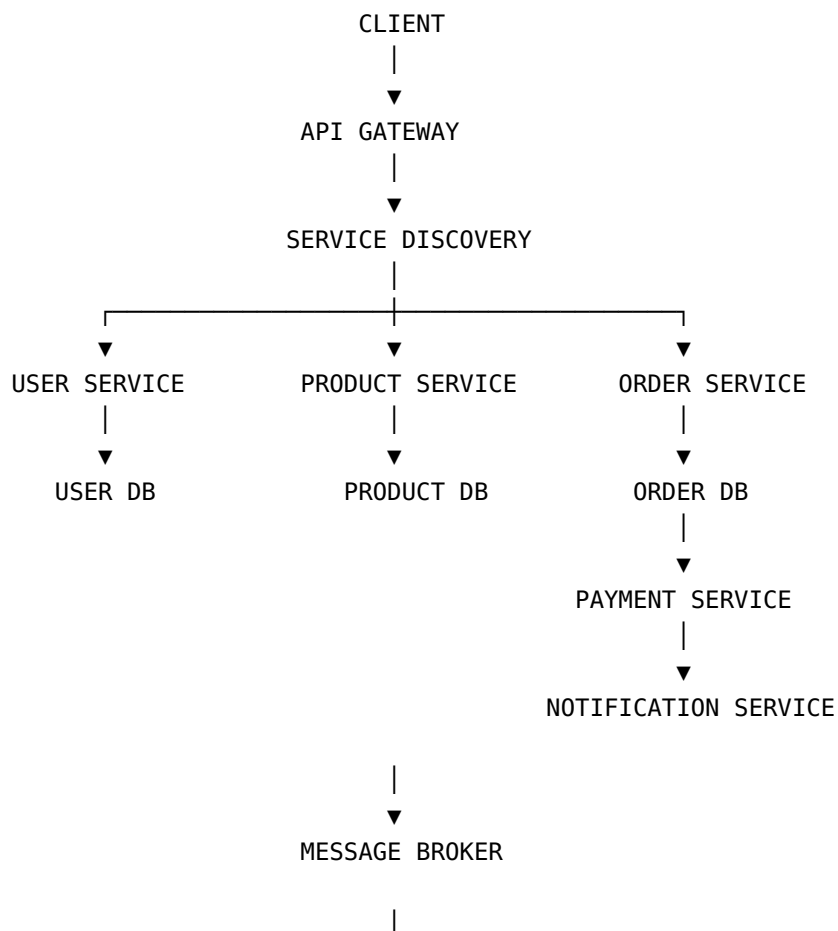
```
docker compose up
```

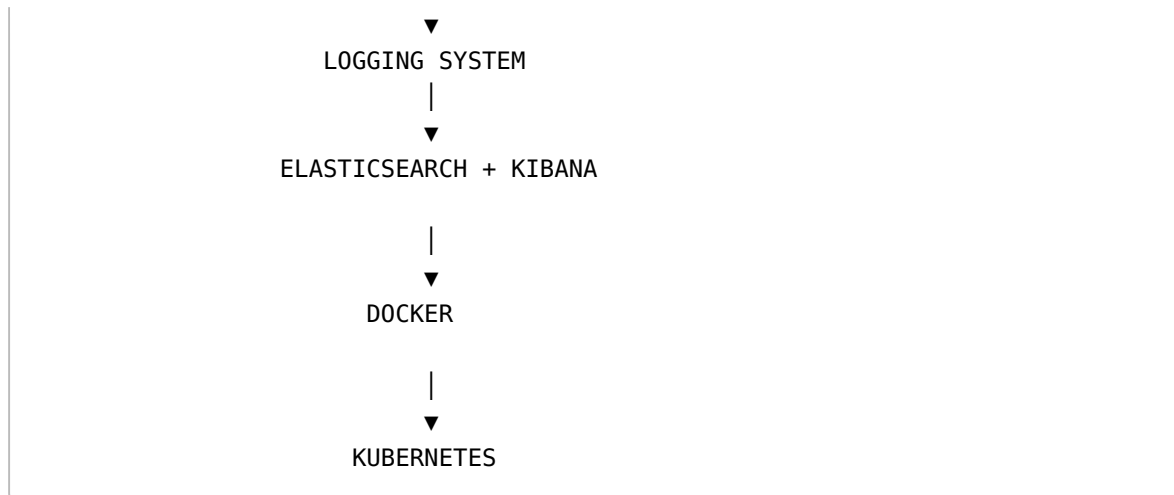
oppure, nella versione Kubernetes:

```
kubectl apply -f kubernetes/
```

e avere un sistema composto da più microservizi funzionanti.

L'architettura finale sarà:





23. Livelli dell'esercizio

Per evitare di costruire tutto contemporaneamente, il progetto deve essere sviluppato in questi livelli:

LIVELLO 1 — Spring Boot

Creare:

```
User Service
Product Service
Order Service
```

con REST API e database.

LIVELLO 2 — Microservizi

Implementare la comunicazione REST tra:

```
Order → User
Order → Product
Order → Payment
```

LIVELLO 3 — Spring Cloud

Aggiungere:

```
Eureka
API Gateway
```

LIVELLO 4 — Comunicazione asincrona

Aggiungere:

```
Kafka oppure RabbitMQ
```

e trasformare alcune comunicazioni da sincrone ad asincrone.

LIVELLO 5 — Docker

Creare:

```
Dockerfile  
docker-compose.yml
```

per l'intero sistema.

LIVELLO 6 — Logging

Implementare:

```
ELK
```

e centralizzare i log.

LIVELLO 7 — Kubernetes

Creare:

```
Deployment  
Service  
ConfigMap  
Secret
```

e distribuire l'applicazione su Kubernetes.

LIVELLO 8 — Scalabilità

Portare:

```
order-service
```

a:

3 replicas

e verificare il funzionamento del sistema.

24. Risultato finale

Il progetto dovrà dimostrare di saper utilizzare insieme:

```
Java
↓
Spring Boot
↓
Spring REST
↓
Spring Data JPA
↓
Microservizi
↓
Spring Cloud
↓
Eureka
↓
API Gateway
↓
Comunicazione REST
↓
Comunicazione asincrona
↓
Kafka/RabbitMQ
↓
Docker
↓
Docker Compose
↓
ELK
↓
Kubernetes
```

Il progetto dovrà inoltre essere gestito con **Git**, utilizzando commit separati per ogni fase dello sviluppo.